

Serving Telemetry for Local LLM Inference: What Production Systems Measure, and How to Recover It from llama.cpp

Luiz Henrique Simões

Independent researcher

luizhenriquesimoes@usp.br · simoeszh@gmail.com

September 2026 (deployment description revised 21 September)

Technical note. Not peer reviewed.

Abstract

Production LLM serving systems, their open standards and the serving literature share one vocabulary of metrics: time to first token timed from request arrival, queue time separate from prefill, per-gap inter-token latency distinct from mean time per output token, goodput as *joint* SLO attainment, current KV-cache occupancy, speculative-decoding acceptance and mean acceptance length, and energy integrated from hardware counters rather than sampled power. This note synthesises those definitions from 16 arXiv papers and 25 specifications, engine documents and source files, all opened during the research. It then applies them to a single-host llama.cpp deployment serving a 27B-parameter reasoning model at a 262,144-token context on two 16 GB consumer GPUs. llama.cpp natively exposes 16 serving series and no latency histogram. It records no request arrival time, so queue time, arrival-based TTFT, inter-token gaps, HTTP errors and finish reasons are unobservable from the server alone. A first deployment built from the server log shows how easily the definitions drift: 4 of its 7 derived rules that were checked never returned a value, and its “TTFT” was prefill time. We describe the adapted stack: a transparent measuring proxy, a server sidecar, an NVML exporter using the GPU performance-monitoring interface on consumer cards, and provisioned dashboards. Each metric is mapped to the standard it follows.

1 Introduction

A serving engine can be tuned only against what is measured, and what is measured depends on definitions that differ more than their shared names suggest. “Time to first token” starts at request arrival in OpenTelemetry [1] and vLLM [2]. The llama.cpp server log starts the same clock only when a slot begins prompt processing [3]. With a single decoding slot, the difference is the entire wait behind the previous request.

This note accompanies a measurement study of quantization, context length and speculative decoding for Qwen3.8-27B on consumer hardware [4]. The study’s host serves that model continuously from a llama.cpp server with a 262,144-token window, one slot, and reasoning enabled by default. The host has two 16 GB consumer GPUs with no GPU interconnect, so the model is split layer-wise across both cards. Host memory is smaller than the model file.

Two served configurations appear in this note. The telemetry was built and verified on 14 September 2026 against the first: a 6-bit GGUF build (UD-Q6_K), a 4-bit (q4_0) KV cache,

and the model’s built-in multi-token-prediction (MTP) drafter with two draft tokens. Since 19 September the host serves the configuration the study selected: a 4-bit build (UD-Q4_K_XL), an 8-bit (q8_0) KV cache, and a separate block-diffusion drafter (DFlash2) with seven draft tokens, on a newer upstream llama.cpp build. Nothing in the design depends on which of the two is running; Section 6 records what was re-checked after the change. All verification figures in Section 5 are from the first configuration and are labelled as such.

This note records three things:

1. How production engines, open standards and the serving literature define serving telemetry.
2. What llama.cpp exposes against that taxonomy.
3. How the definitions were adapted for this deployment, including the defects a first, log-based attempt contained.

Method. Three parallel research passes each covered one area: (A) the llama.cpp telemetry surface; (B) serving standards, engine metric catalogues and the serving literature; and (C) hardware, energy, host and output-behaviour telemetry together with monitoring method. Each pass opened its sources and marked every source either as fetched or as recalled. Only fetched sources are cited here. arXiv metadata was re-checked against the arXiv API. Claims about the running server were checked read-only against the live endpoints and the engine source.

2 How serving telemetry is defined

2.1 Latency is decomposed at request boundaries

TTFT. Every standard examined times the first token from request arrival or send, so queueing is included:

- vLLM’s `time_to_first_token_seconds` starts at frontend arrival [2];
- OpenTelemetry’s `gen_ai.server.time_to_first_token` is described as covering “the time spent in the queue and the prefill phase” [1];
- NVIDIA AIPerf times from request start [5];
- MLPerf Inference judges TTFT per query at a latency percentile [6].

Queue and prefill. vLLM separates queue time (queued $\rightarrow$ scheduled) from prefill time (scheduled $\rightarrow$ first token) [2], and TGI exports a queue-duration histogram [7].

Decode. Two quantities are distinguished:

- *Time per output token* (TPOT) is a per-request mean, the decode time divided by $n_{\text{out}} - 1$ (OpenTelemetry, vLLM, MLPerf).
- *Inter-token latency* (ITL) records one observation per gap between emitted tokens or chunks (vLLM, NVIDIA Dynamo [8], AIPerf).

The distinction matters under speculative decoding, where tokens arrive in bursts: a mean TPOT hides a bimodal gap distribution.

Reasoning models. For reasoning models AIPerf adds time to first *output* (non-reasoning) token [5]. OpenAI’s API reports reasoning tokens separately, and documents responses that end before any visible output [9].

Streaming experience. Etalon’s fluidity-index [10] and Andes’ quality-of-experience metric [11] score the timeline of token delivery rather than single percentiles.

2.2 Goodput is joint SLO attainment

DistServe defines SLO attainment as the proportion of requests that meet their SLOs, and goodput as the highest request rate served at a target attainment [12]. AIPerf reports goodput as good requests per second [5]. In both, a request is judged against *all* its objectives at once. Sarathi-Serve frames the throughput–latency trade-off that goodput is meant to expose [13]. Splitwise [14] and DistServe both motivate recording prefill and decode as separate phases.

2.3 Saturation is the queue, the running set and the KV cache

The Kubernetes Gateway API Inference Extension specifies the load-balancing contract a model server must export. It has three gauges: queued requests, running requests and KV-cache utilisation [15]. vLLM and SGLang map these to their own metric names [2, 16].

Beyond that contract:

- **KV-cache occupancy.** PagedAttention made occupancy the central memory quantity [17].
- **Prefix-cache hits.** These are counted in tokens, as hits over queries, by vLLM and SGLang [2, 16]. They are the dominant variable for KV-centric designs such as Mooncake [18].
- **Scheduling.** Llumnix treats queueing and migration as the levers [19].

2.4 Speculative decoding has three quantities, not one

Three quantities are distinguished:

- **Acceptance rate,** accepted draft tokens over drafted tokens.
- **Per-token acceptance probability** α , from which the expected tokens per step follow as $(1 - \alpha^{\gamma+1})/(1 - \alpha)$ for draft length γ [20].
- **Mean acceptance length,** $\tau = 1 + \text{accepted}/\text{drafts}$, the quantity EAGLE reports [21] and vLLM computes [2].

Per-position acceptance counters give the conditional α_k , which separates a drafter failure from an effect of position. Wall-clock speedup requires a paired no-speculation baseline [20, 22, 23].

2.5 Hardware utilisation and energy need the right instrument

Utilisation. NVML’s classic utilisation is the fraction of the sample period during which *at least one* kernel ran [24]. DCGM’s profiling metrics instead report streaming-multiprocessor activity and occupancy, memory-bandwidth activity and PCIe throughput [25].

Table 1: Serving-telemetry taxonomy and the definition each family follows.

family	metrics	defined by
latency	TTFT (arrival), time to first answer token, queue, prefill, TPOT, ITL per gap, E2E	[1, 2, 5, 6]
goodput	joint SLO attainment; good requests/s	[5, 12]
saturation	queued, running, KV-cache occupancy, prefix-cache hit ratio	[2, 15, 16]
speculation	acceptance rate, τ , per-position α_k	[2, 20, 21]
errors	HTTP class, aborts, finish reason, empty answers	[9, 32, 37]
request shape	ISL (incl. cached), OSL, reasoning tokens, requested budget	[5, 35, 36]
GPU	SM activity/occupancy, memory bandwidth, PCIe, energy counter	[24, 25]
energy	J/token and J/request from counters, idle power, Wh/1k requests	[29–31]
host	PSI, swap, faults, CPU package energy	[33, 34]

Energy. Energy should come from the cumulative counter, not from integrated power samples:

- `nvidia-smi` power readings cover only part of runtime on common data-centre GPUs, and correcting for this materially reduces energy error [26].
- Zeus and the ML.ENERGY benchmark measure energy per request and per response from counters [27–29].
- The AI Energy Score normalises to Wh per 1,000 queries [30].
- Google’s accounting of production inference counts idle provisioned capacity as a separate term [31].

2.6 Method: golden signals, USE, workload shape

Three frameworks organise the dashboards. The four golden signals are latency, traffic, errors and saturation, and they count a successful response with wrong content as an error [32]. The USE method (utilisation, saturation, errors) applies per resource [33]. Pressure-stall information quantifies host saturation [34].

Workload traces from Azure [35] and BurstGPT [36] show which request-shape distributions matter: input and output length, and inter-arrival burstiness.

3 What llama.cpp exposes

The server’s Prometheus endpoint exports 16 series: 10 counters, 5 gauges and one per-position speculative-decoding counter. There are no histograms and no KV-occupancy gauge. The engine source and its issue history [3, 38] show five properties that shape everything downstream.

1. **No arrival time is recorded.** The per-request clock starts when a slot begins prompt processing, after HTTP parsing, templating, tokenisation and the deferred queue. The logged “total time” is prefill plus decode.
2. **The KV-occupancy gauge was removed.** It was judged too implementation-specific to expose, and a request to restore it has stalled [38]. What remains, `n_tokens_max`, is a lifetime high-water mark.

3. **Two gauges reset whenever /metrics is read.** The throughput gauges average over the window since the previous scrape by any reader, so any second reader corrupts them. Decode and speculation counters advance only when a request ends.
4. **truncated is not length.** The flag is set only when the context window fills. Exhausting `max_tokens` yields `finish_reason: length` with `truncated` false.
5. **A lot is available per request, but only through the response or the slot endpoint.** Each response carries a `timings` object (prompt, cached and decoded tokens with their durations, drafted and accepted tokens). `/slots` reports live per-slot prompt, cache and decode progress.

4 Adaptation for this deployment

4.1 Constraints

Six properties of the deployment constrain the design:

- **One decoding slot**, so any concurrent request queues, and any synthetic probe request delays a real one.
- **Reasoning on by default**, so the first token is usually a reasoning token.
- **A full-window prefill takes minutes**, so fixed TTFT SLOs are meaningless at depth.
- **Speculative decoding emits tokens in bursts.**
- **Consumer GPUs**: NVIDIA does not document DCGM profiling fields for them.
- **Memory headroom is small**, which rules out tracing and log-store backends.

4.2 Architecture

Figure 1 shows the stack. Its five components are:

- **A transparent measuring proxy.** It takes over the address clients already use, and the server moves to loopback. It relays every path byte-for-byte and streams chunk by chunk. For inference endpoints it timestamps arrival, the first token-bearing chunk, the first answer chunk, every chunk gap and the last byte. It parses `finish_reason`, `usage` and the server's `timings`, evaluates SLO classes per request, and writes one content-free JSON record per request.
- **A server sidecar.** It polls `/health`, `/slots` and `/props`; it never polls `/metrics`, because of property 3. It follows the container log with wall-clock timestamps and exports launch-configuration provenance as info labels.
- **An NVML exporter.** It reads the cumulative energy counter, the GPU performance-monitoring (GPM) ratios for streaming-multiprocessor utilisation, occupancy and memory bandwidth, PCIe throughput, the PCIe replay counter, XID events and per-process memory attributed to containers. Both consumer cards answered every one of these queries.
- **Recording rules and alerts** following Table 1.
- **Four provisioned dashboards**: golden signals, request anatomy, GPU and energy (USE per card), and host.

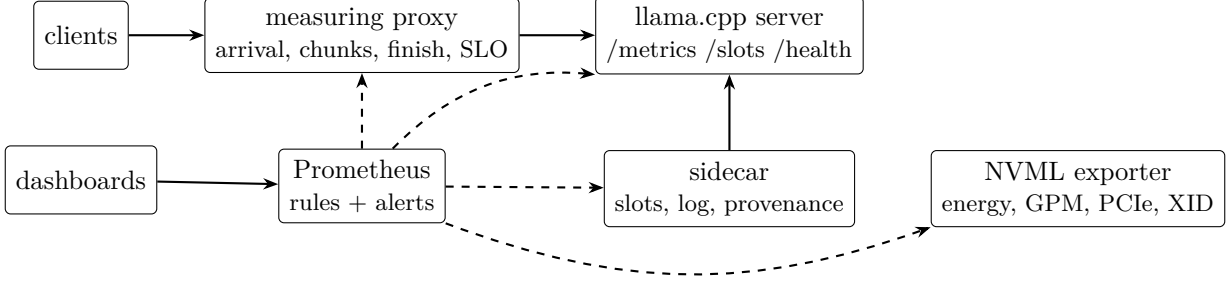


Figure 1: Adapted telemetry stack. Solid arrows: request or poll paths. Dashed arrows: Prometheus scrapes. Only the proxy sits in the request path.

Table 2: Standard quantity and how it is observed on llama.cpp in this deployment.

quantity	observation
TTFT	proxy: arrival $\rightarrow$ first token-bearing chunk (streaming)
time to first answer token	proxy: arrival $\rightarrow$ first non-reasoning content chunk
queue / prefill	proxy TTFT $- \text{timings.prompt_ms} / \text{timings.prompt_ms}$
TPOT / ITL	$\text{predicted_ms} / (n - 1) / \text{proxy chunk gaps}$
goodput	per-request joint predicate $\rightarrow$ good and evaluated counters
finish reason, errors, aborts	proxy: response body, HTTP status, client disconnect
ISL, cached, reasoning tokens	$\text{prompt_n} + \text{cache_n}$; cache_n ; reasoning chunks or tokenisation
KV occupancy	sidecar: slot prompt + decoded tokens over slot context
acceptance, τ , α_k	native counters: accepted/drafted; $1 + \text{accepted/drafts}$; per-position ratios
energy per token / request	Δ NVML energy counter / Δ tokens or requests, traffic-gated
GPU utilisation	NVML GPM streaming-multiprocessor utilisation and occupancy

4.3 Mapping definitions to observations

Table 2 states how each standard quantity is obtained. Two quantities are approximations, and the approximation is recorded in the metric description:

- **Queue time** is TTFT minus server prefill time, so it includes templating and tokenisation.
- **TTFT for non-streaming requests** is end-to-end time minus server decode time.

The SLO thresholds are specific to the host and are not MLPerf targets. Two classes are evaluated. An *interactive* class requires $\text{TTFT} \leq 8\text{ s}$ and $\text{TPOT} \leq 75\text{ ms}$. A *depth-normalised* class requires $\text{queue} \leq 2\text{ s}$, $\text{prefill} \leq 2\text{ s} + 2.5\text{ ms}$ per uncached prompt token, and $\text{TPOT} \leq 100\text{ ms}$.

5 Lessons from a first deployment

Before this work, a log-tailing exporter and a set of recording rules had been deployed. Checking them against the definitions above found the following.

- **Mislabelled latency.** “TTFT” was prefill time, and “end-to-end” was prefill plus decode.
- **A wrong truncation signal.** It used the context-window flag, so budget exhaustion, which is the failure mode that matters for a reasoning model, was counted as a normal stop.

- **A goodput rule that could never evaluate.** Prometheus 3 stores the histogram bucket bound 8 as 8.0. The rule was also a product of marginal fractions rather than joint attainment.
- **Energy rules that could never evaluate** because of label mismatches. Once repaired, they divided an instantaneous power sample by a five-minute token rate, which gives tens of kilojoules per token at idle.
- **A “KV utilisation” that was a lifetime maximum.**
- **A biased decode rate**, which divided n tokens by $n - 1$ steps of decode time.

None of these is exotic: every item follows from assuming a metric means what its name suggests. A test that queries every recording rule while traffic exists would have caught four of them at once, and the adapted stack includes such a check.

Validation. After cut-over, a battery of eight small requests exercised the stack:

- non-streaming and streaming requests;
- reasoning on and off;
- budget exhaustion;
- two concurrent requests;
- a client disconnect;
- a non-inference 404.

Every check behaved as specified:

- **Reasoning split.** Streaming with reasoning on gave a TTFT of 1.10s but a time to first *answer* token of 9.40s: 176 reasoning chunks, 78 answer chunks and 268 inter-chunk gaps.
- **Chunks are tokens.** Counting streamed reasoning chunks as tokens had been checked beforehand: 87 chunks tokenised to exactly 87 tokens.
- **Budget exhaustion.** A 16-token budget ended as `length` and was counted as an empty answer.
- **Queueing.** Of two concurrent requests, the second waited 3.32s against 0.87s. It met the interactive SLO but failed the depth-normalised queue clause, so attainment for that class read 0.75.
- **Disconnects.** A client disconnect was recorded as an abort, and upstream generation was cancelled within 0.25s.
- **Non-inference paths.** A non-inference 404 passed through and was not counted.

Rule and dashboard coverage. 44 of the 69 rules and alerts returned data. The other 25 were empty for stated reasons:

- 18 were alerts not firing;
- 6 were conditional-acceptance positions that a two-token drafter cannot reach (the drafter in service when this was verified; see Section 6);
- 1 was an idle-power helper, deliberately empty while traffic is recent.

All 156 dashboard queries parsed, and 150 returned data.

Hardware and cost. On the consumer GPUs, GPM streaming-multiprocessor utilisation peaked at 0.50 and 0.44 during a 256-token decode, with memory-bandwidth utilisation at 0.35 and 0.38. The energy-counter rate agreed with sampled power within 0.3 % at idle and within 7.2 % under a short burst. The proxy added a median 0.12 ms on a non-inference path. The new processes used 47, 28 and 17 MiB of resident memory.

Two further defects surfaced during this validation, both instances of the same lesson.

- **A series born on its first event is invisible to rate functions.** It is first scraped already at 1, so `rate()` and `increase()` never see the step. The first abort, the first length stop, and the first time-to-first-answer-token observation of each label combination were lost, and the corresponding quantile rule stayed empty. The fix is to create the expected label combinations at zero on start-up.
- **Joins broke during a configuration reload.** A per-slot join failed briefly because two scrape jobs exported the same series. Aggregating per slot before dividing makes the rule robust to that.

Queue floor. At idle the queue-time estimate is not zero: it read 0.18–0.65 s on streaming requests. It includes templating, tokenisation and slot selection before the server’s prefill clock starts. That time is real user-visible latency, but it is not queueing in the scheduling sense.

6 After the served configuration changed

On 19 September 2026 the served configuration changed in four respects at once: weight build, KV-cache precision, drafter (from an in-checkpoint MTP head to a separate DFlash2 model) and engine build. No telemetry component was modified for it. A read-only check of the metrics store on 21 September found:

- **Launch provenance followed the change on its own.** The sidecar’s launch-information series reports the new cache type (`q8_0`), speculative type (`draft-dflash`), maximum draft length (7), context size, split and engine image, and the server-information series reports the new engine build and model file. This is the purpose of exporting the launch line as labels: a dashboard can never silently describe a configuration that is no longer running.
- **Per-position acceptance now extends past position two.** With a seven-token drafter the per-position counters have reported positions zero to three over the past week, so the conditional-acceptance rules that were structurally empty under the two-token drafter can now return data. Positions four to six had not yet reported; whether that is traffic or a counter limit was not investigated.

- **One label went stale.** The human-readable `model` label still carried the previous configuration’s alias while the model path beside it showed the new file. The alias comes from a static target list, not from the server. It is the same lesson as Section 5 in miniature: a label typed by hand is a claim to verify, and provenance should be read from the running process wherever that is possible. The sidecar was changed the same day to take the alias from the server’s own properties endpoint, keeping the configured value only until the server answers.

The rule-coverage and hardware figures of Section 5 were not re-measured under the new configuration. Acceptance ratios are not comparable across the two drafters, because they divide by different draft lengths; mean accepted length is the comparable quantity, as the accompanying report discusses.

7 Limitations

- **Wall power is unmeasurable** on this host. Energy covers GPU counters and CPU package energy only.
- **GPM ratios on consumer GPUs are undocumented by NVIDIA.** They were validated only against known load.
- **The queue-time approximation** includes templating and tokenisation, and reads 0.18–0.65 s even when nothing is queued.
- **Validation used a handful of short requests.** It shows that the instruments record what they claim. It is not a performance characterisation, and no latency or energy figure here should be read as one.
- **Inter-token gaps are measured per streamed event**, so tokens emitted inside one speculative burst cannot be separated.
- **Alerts have no notification channel** and surface only in dashboards.
- **No quality canary is run:** with one slot, a canary would delay user requests.
- **This is one host, one engine build and one model.** The mapping is not validated against other `llama.cpp` versions or router deployments.
- **This note is not peer reviewed.**

8 Conclusion

The serving community has converged on precise, decomposed definitions: arrival-based latency, joint goodput, current saturation, counter-based energy. A local `llama.cpp` deployment can recover nearly all of them, but not from the server alone. The missing piece is the request boundary, which only a component in the request path can see. The server’s own per-request **timings** and slot state supply the rest. The defects found in a first deployment argue for treating every metric definition as a claim to verify against its standard, not a name to trust.

References

- [1] OpenTelemetry. Semantic conventions for generative AI metrics. Specification (Development status), 2026. URL <https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-metrics.md>.
- [2] vLLM project. vLLM metrics: Design document and production metrics, 2026. URL <https://docs.vllm.ai/en/latest/design/metrics.html>.
- [3] ggml-org. llama.cpp HTTP server: Documentation and source (tools/server), 2026. URL <https://github.com/ggml-org/llama.cpp/tree/master/tools/server>.
- [4] Luiz Henrique Simões. What benchmarks miss: What quantization and speculative decoding really cost Qwen3.8-27B in accuracy, context length and generation speed. <https://github.com/henrique-simoes/llm-quantization-damage>, 2026. Technical report, not peer reviewed.
- [5] NVIDIA. AIPerf metrics reference, 2026. URL <https://docs.nvidia.com/aiperf/reference/ai-perf-metrics-reference>.
- [6] MLCommons. MLPerf inference rules, 2026. URL https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc.
- [7] Hugging Face. Text generation inference: Metrics, 2025. URL <https://huggingface.co/docs/text-generation-inference/reference/metrics>.
- [8] NVIDIA. Dynamo metrics catalog, 2026. URL <https://docs.nvidia.com/dynamo/reference/observability/metrics-catalog>.
- [9] OpenAI. Reasoning models. API documentation, 2026. URL <https://developers.openai.com/api/docs/guides/reasoning>.
- [10] Amey Agrawal et al. Etalon: Holistic performance evaluation framework for LLM inference systems, 2024. arXiv:2407.07000.
- [11] Jiachen Liu et al. Andes: Defining and enhancing quality-of-experience in LLM-based text streaming services, 2024. arXiv:2404.16283.
- [12] Yinmin Zhong et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024. arXiv:2401.09670.
- [13] Amey Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, 2024. arXiv:2403.02310.
- [14] Pratyush Patel et al. Splitwise: Efficient generative LLM inference using phase splitting, 2023. arXiv:2311.18677.
- [15] Kubernetes SIG Network, gateway-api-inference-extension. Model server protocol for Gateway API inference extension (proposal 003), 2025. URL <https://github.com/kubernetes-sigs/gateway-api-inference-extension/blob/main/docs/proposals/003-model-server-protocol/README.md>.
- [16] SGLang project. SGLang production metrics, 2026. URL https://docs.sglang.io/references/production_metrics.html.

- [17] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention, 2023. arXiv:2309.06180.
- [18] Ruoyu Qin et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024. arXiv:2407.00079.
- [19] Biao Sun et al. Llumnix: Dynamic scheduling for large language model serving, 2024. arXiv:2406.03243.
- [20] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2022. arXiv:2211.17192.
- [21] Yuhui Li et al. EAGLE: Speculative sampling requires rethinking feature uncertainty, 2024. arXiv:2401.15077.
- [22] Charlie Chen et al. Accelerating large language model decoding with speculative sampling, 2023. arXiv:2302.01318.
- [23] Tianle Cai et al. Medusa: Simple LLM inference acceleration framework with multiple decoding heads, 2024. arXiv:2401.10774.
- [24] NVIDIA. NVML API reference: Device queries, 2026. URL https://docs.nvidia.com/deploy/nvml-api/latest/api/group__nvmlDeviceQueries.html.
- [25] NVIDIA. DCGM feature overview: Profiling metrics, 2026. URL <https://docs.nvidia.com/datacenter/dcgm/latest/user-guide/feature-overview.html>.
- [26] Zeyu Yang, Karel Adamek, and Wesley Armour. Part-time power measurements: nvidia-smi’s lack of attention, 2023. arXiv:2312.02741.
- [27] ML.ENERGY Initiative. Zeus: Measuring GPU energy, 2026. URL <https://ml.energy/zeus/measure/>.
- [28] Jae-Won Chung. Measuring GPU energy: Best practices. ML.ENERGY blog, 2023. URL <https://ml.energy/blog/energy/measurement/measuring-gpu-energy-best-practices/>.
- [29] Jae-Won Chung et al. The ML.ENERGY benchmark: Toward automated inference energy measurement and optimization, 2025. arXiv:2505.06371.
- [30] Hugging Face. AI energy score, 2025. URL <https://huggingface.github.io/AIEnergyScore/>.
- [31] Cooper Elsworth et al. Measuring the environmental impact of delivering AI at Google scale, 2025. arXiv:2508.15734.
- [32] Rob Ewaschuk. Monitoring distributed systems. In Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors, *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. URL <https://sre.google/sre-book/monitoring-distributed-systems/>.
- [33] Brendan Gregg. The USE method, 2012. URL <https://www.brendangregg.com/usemethod.html>.

- [34] The Linux Kernel Documentation. PSI: Pressure stall information, 2026. URL <https://docs.kernel.org/accounting/psi.html>.
- [35] Microsoft Azure. Azure LLM inference dataset 2023, 2023. URL <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md>.
- [36] Yuxin Wang et al. BurstGPT: A real-world workload dataset to optimize LLM serving systems, 2024. arXiv:2401.17644.
- [37] OpenTelemetry. Semantic conventions for generative AI spans, 2026. URL <https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-spans.md>.
- [38] ggml-org/llama.cpp contributors. Feature request: Expose KV cache utilization for /metrics endpoint (issue 23632), 2026. URL <https://github.com/ggml-org/llama.cpp/issues/23632>.